

# SSM整合

鸣谢：黑马程序员



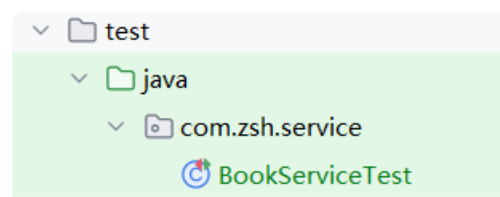
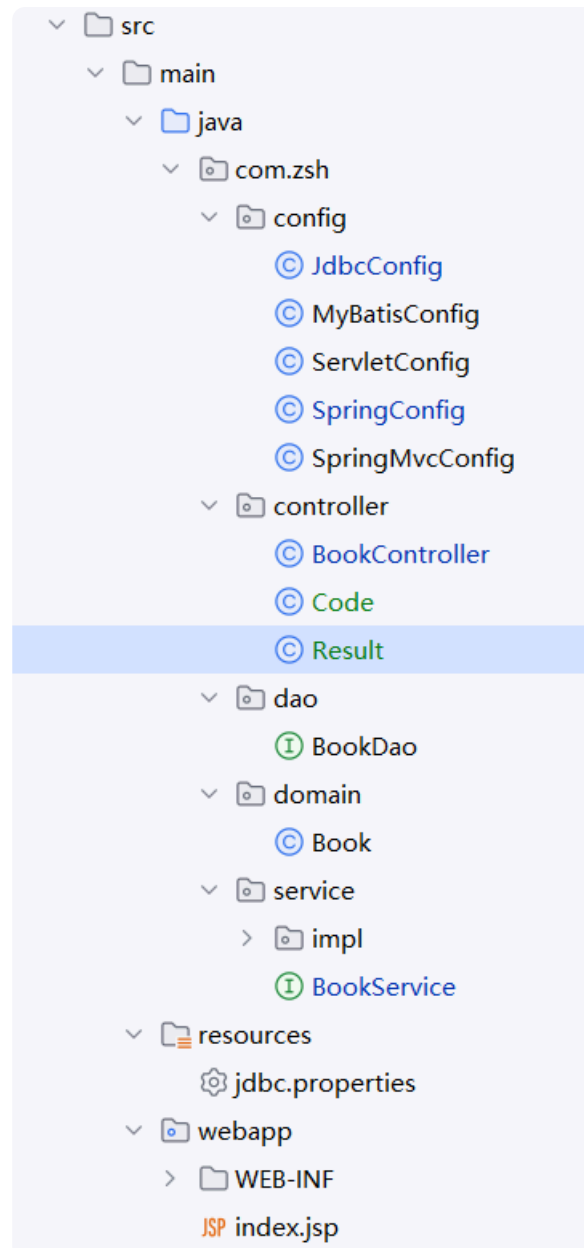
黑马程序员SSM框架教程\_Spring+SpringMVC+Maven高级+SpringBo...

UP 黑马程序员 · 2022-4-20

项目结构：

```
1  src
2  |--main
3  |   |--java
4  |       |--com
5  |           |--zsh
6  |               |--config
7  |                   JdbcConfig.java
8  |                   MyBatisConfig.java
9  |                   ServletConfig.java
10 |                   SpringConfig.java
11 |                   SpringMvcConfig.java
12 |
13 |               |--controller
14 |                   BookController.java
15 |                   Code.java
16 |                   Result.java
17 |
18 |               |--dao
19 |                   BookDao.java
20 |
```

```
21 | | | | |--domain
22 | | | | | Book.java
23 | | | | |
24 | | | | |--service
25 | | | | | BookService.java
26 | | | | |
27 | | | | |--impl
28 | | | | | BookServiceImpl.java
29 | | | |
30 | | |--resources
31 | | | jdbc.properties
32 | | |
33 | |--webapp
34 | | index.jsp
35 | |
36 | | |--WEB-INF
37 | | | web.xml
38 | |
39 |--test
40 | |--java
41 | | |--com
42 | | | |--zsh
43 | | | | |--service
44 | | | | | BookServiceTest.java
```



# Part I 后端开发

## 一、SSM整合流程

1. 创建工程

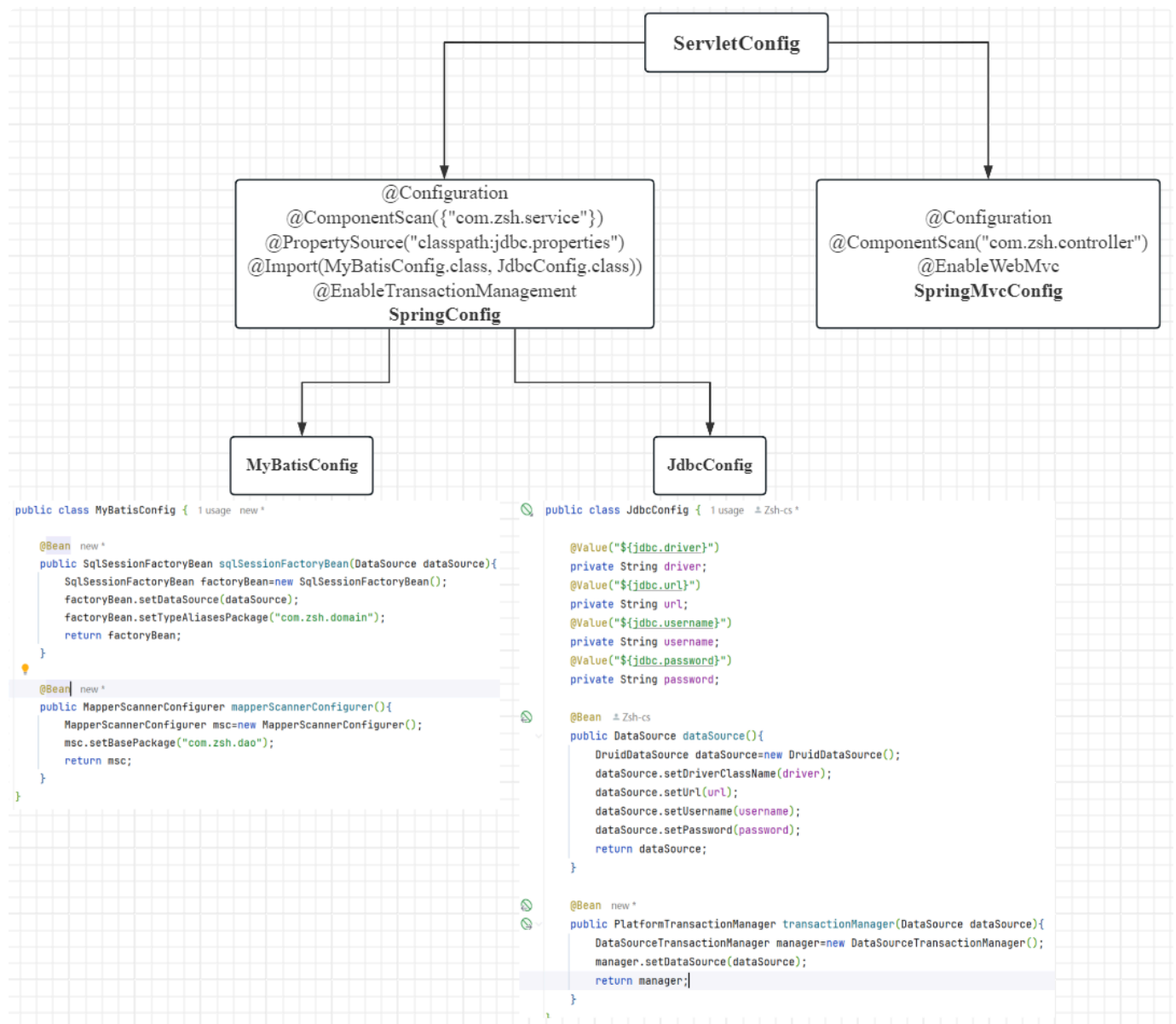
2. SSM整合配置：

- Spring: `SpringConfig`
- SpringMVC:
  - `ServletConfig`
  - `SpringMvcConfig`
- MyBatis:
  - `MyBatisConfig`
  - `JdbcConfig`
  - `jdbc.properties`

3. 功能模块开发：

- 表与实体类
  - dao（接口+自动代理）
  - service（接口+实现类）：业务层接口测试（整合 `JUnit`）
  - controller：表现层接口测试（`Postman`）
-

## 二、`config`包—核心配置类



### 1. `jdbc.properties` + `JdbcConfig`

- `jdbc.properties`:

```
1 jdbc.driver=com.mysql.cj.jdbc.Driver  
2 jdbc.url=jdbc:mysql://localhost:3306/ssm_db  
3 jdbc.username=root  
4 jdbc.password=123456
```

- `JdbcConfig`:

```
1 package com.zsh.config;  
2
```

```
3 import com.alibaba.druid.pool.DruidDataSource;
4 import org.springframework.beans.factory.annotation.Value;
5 import org.springframework.context.annotation.Bean;
6
7 import javax.sql.DataSource;
8
9 public class JdbcConfig {
10
11     @Value("${jdbc.driver}")
12     private String driver;
13     @Value("${jdbc.url}")
14     private String url;
15     @Value("${jdbc.username}")
16     private String username;
17     @Value("${jdbc.password}")
18     private String password;
19
20     @Bean
21     public DataSource dataSource(){
22         DruidDataSource dataSource=new DruidDataSource();
23         dataSource.setDriverClassName(driver);
24         dataSource.setUrl(url);
25         dataSource.setUsername(username);
26         dataSource.setPassword(password);
27         return dataSource;
28     }
29
30     @Bean
31     public PlatformTransactionManager transactionManager(DataSource
dataSource){
32         DataSourceTransactionManager manager=new
DataSourceTransactionManager();
33         manager.setDataSource(dataSource);
34         return manager;
35     }
36 }
```

## 2. MyBatisConfig

```
1 package com.zsh.config;
2
3 import org.mybatis.spring.SqlSessionFactoryBean;
4 import org.mybatis.spring.mapper.MapperScannerConfigurer;
5 import org.springframework.context.annotation.Bean;
6
7 import javax.sql.DataSource;
8
9 public class MyBatisConfig {
10
11     @Bean
12     public SqlSessionFactoryBean sqlSessionFactoryBean(DataSource
13     dataSource){
14         SqlSessionFactoryBean factoryBean=new SqlSessionFactoryBean();
15         factoryBean.setDataSource(dataSource);
16         factoryBean.setTypeAliasesPackage("com.zsh.domain");
17         return factoryBean;
18     }
19
20     @Bean
21     public MapperScannerConfigurer mapperScannerConfigurer(){
22         MapperScannerConfigurer msc=new MapperScannerConfigurer();
23         msc.setBasePackage("com.zsh.dao");
24         return msc;
25     }
26 }
```

## 3. SpringConfig

```
1 package com.zsh.config;
2
3 import org.springframework.context.annotation.ComponentScan;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.context.annotation.Import;
6 import org.springframework.context.annotation.PropertySource;
7
8 @Configuration
9 @ComponentScan({"com.zsh.service"})
```

```

10 @PropertySource("classpath:jdbc.properties")
11 @Import({MyBatisConfig.class, JdbcConfig.class})
12 @EnableTransactionManagement
13 public class SpringConfig {
14 }

```

## 4. SpringMvcConfig

```

1 package com.zsh.config;
2
3 import org.springframework.context.annotation.ComponentScan;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.web.servlet.config.annotation.EnableWebMvc;
6
7 @Configuration
8 @ComponentScan("com.zsh.controller")
9 @EnableWebMvc
10 public class SpringMvcConfig {
11 }

```

## 5. ServletConfig

```

1 package com.zsh.config;
2
3 import org.springframework.web.filter.CharacterEncodingFilter;
4 import
5     org.springframework.web.servlet.support.AbstractAnnotationConfigDispatch
6     herServletInitializer;
7
8 import javax.servlet.Filter;
9
10 public class ServletConfig extends
11     AbstractAnnotationConfigDispatcherServletInitializer {
12     @Override
13     protected Class<?>[] getRootConfigClasses() {
14         return new Class[]{SpringConfig.class};
15     }
16 }

```



```
13
14     @Override
15     protected Class<?>[] getServletConfigClasses() {
16         return new Class[]{SpringMvcConfig.class};
17     }
18
19     @Override
20     protected String[] getServletMappings() {
21         return new String[]{"/*"};
22     }
23
24     // POST请求中文乱码处理：设置过滤器
25     @Override
26     protected Filter[] getServletFilters() {
27         CharacterEncodingFilter filter=new CharacterEncodingFilter();
28         filter.setEncoding("UTF-8");
29         return new Filter[]{filter};
30     }
31 }
```

---

## 三、功能模块开发

### 1. Book

```
1 package com.zsh.domain;
2
3 public class Book {
4     private Integer id;
5     private String type;
6     private String name;
7     private String description;
8
9     public Book(){}
10
11     public Book(Integer id, String type, String name, String
description) {
12         this.id = id;
13         this.type = type;
```

```

14         this.name = name;
15         this.description = description;
16     }
17
18     @Override
19     public String toString() {
20         return "Book{" +
21             "id=" + id +
22             ", type='" + type + '\'' +
23             ", name='" + name + '\'' +
24             ", description='" + description + '\'' +
25             '}';
26     }
27
28     // 省略getter&setter
29
30 }

```

## 2. BookDao

```

1  package com.zsh.dao;
2
3  import com.zsh.domain.Book;
4  import org.apache.ibatis.annotations.*;
5  import org.springframework.stereotype.Repository;
6
7  import java.util.List;
8
9  @Repository
10 public interface BookDao {
11
12     @Insert("insert into books (type, name, description) value (#
13     {type},#{name},#{description})")
14     void save(Book book);
15
16     @Delete("delete from books where id=#{id}")
17     void delete(Integer id);
18
19     @Update("update books set type=#{type},name=#{name},description=#
20     {description} where id=#{id}")
21     void update(Book book);

```

```

20
21     @Select("select * from books where id=#{id}")
22     Book getById(Integer id);
23
24     @Select("select * from books")
25     List<Book> getAll();
26 }

```

### 3. BookService + BookServiceImpl + BookServiceTest

- BookService:

```

1  package com.zsh.service;
2
3  import com.zsh.domain.Book;
4  import org.springframework.transaction.annotation.Transactional;
5
6  import java.util.List;
7
8  @Transactional
9  public interface BookService {
10     boolean save(Book book);
11     boolean delete(Integer id);
12     boolean update(Book book);
13     Book getById(Integer id);
14     List<Book> getAll();
15 }

```

- BookServiceImpl:

```

1  package com.zsh.service.impl;
2
3  import com.zsh.dao.BookDao;
4  import com.zsh.domain.Book;
5  import com.zsh.service.BookService;
6  import org.springframework.beans.factory.annotation.Autowired;
7  import org.springframework.stereotype.Service;
8
9  import java.util.List;
10
11 @Service

```

```

12 public class BookServiceImpl implements BookService {
13
14     @Autowired
15     private BookDao bookDao;
16
17     @Override
18     public boolean save(Book book) {
19         bookDao.save(book);
20         return true;
21     }
22
23     @Override
24     public boolean delete(Integer id) {
25         bookDao.delete(id);
26         return true;
27     }
28
29     @Override
30     public boolean update(Book book) {
31         bookDao.update(book);
32         return true;
33     }
34
35     @Override
36     public Book getById(Integer id) {
37         return bookDao.getById(id);
38     }
39
40     @Override
41     public List<Book> getAll() {
42         return bookDao.getAll();
43     }
44 }

```

- **BookServiceTest:**

```

1 package com.zsh.service;
2
3 import com.zsh.config.SpringConfig;
4 import com.zsh.domain.Book;
5 import org.junit.Test;
6 import org.junit.runner.RunWith;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.test.context.ContextConfiguration;

```

```

9  import
   org.springframework.test.context.junit4.SpringJUnit4ClassRunner;
10
11  import java.util.List;
12
13  @RunWith(SpringJUnit4ClassRunner.class)
14  @ContextConfiguration(classes = SpringConfig.class)
15  public class BookServiceTest {
16
17      @Autowired
18      private BookService bookService;
19
20      @Test
21      public void testGetById(){
22          Book book = bookService.getById(1);
23          System.out.println(book);
24      }
25
26      @Test
27      public void testGetAll(){
28          List<Book> books=bookService.getAll();
29          System.out.println(books);
30      }
31  }

```

## 4. BookController

```

1  package com.zsh.controller;
2
3  import com.zsh.domain.Book;
4  import com.zsh.service.BookService;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.web.bind.annotation.*;
7
8  import java.util.List;
9
10 @RestController
11 @RequestMapping("/books")
12 public class BookController {
13
14     @Autowired

```

```
15     private BookService bookService;
16
17     @PostMapping
18     public boolean save(@RequestBody Book book) {
19         return bookService.save(book);
20     }
21
22     @DeleteMapping("/{id}")
23     public boolean delete(@PathVariable Integer id) {
24         return bookService.delete(id);
25     }
26
27     @PutMapping
28     public boolean update(@RequestBody Book book) {
29         return bookService.update(book);
30     }
31
32     @GetMapping("/{id}")
33     public Book getById(@PathVariable Integer id) {
34         return bookService.getById(id);
35     }
36
37     @GetMapping
38     public List<Book> getAll() {
39         return bookService.getAll();
40     }
41
42 }
```

---

## Part II 表现层与前端数据交互

### 一、设置统一的数据返回结果类 `Result`

 Tip

`Result` 类中的字段并不是固定的，可以根据需要自行增减，同时要提供若干个构造方法，方便操作。

```
1 package com.zsh.controller;
2
3 public class Result {
4     private Integer code; // 状态码
5     private Object data; // 数据
6     private String msg; // 响应消息
7
8     public Result() {
9
10    }
11
12    public Result(Integer code, Object data) {
13        this.code = code;
14        this.data = data;
15    }
16
17    public Result(Integer code, Object data, String msg) {
18        this.code = code;
19        this.data = data;
20        this.msg = msg;
21    }
22
23    // 省略getter&setter
24 }
```

## 二、设置统一的数据返回结果状态码类 `Code`

```
1 package com.zsh.controller;
2
3 public class Code {
4     // 结尾1代表成功，0代表失败
5
6     public static final Integer SAVE_OK = 20011;
7     public static final Integer DELETE_OK = 20021;
8     public static final Integer UPDATE_OK = 20031;
9     public static final Integer GET_OK = 20041;
```

```
10
11     public static final Integer SAVE_ERR = 20010;
12     public static final Integer DELETE_ERR = 20020;
13     public static final Integer UPDATE_ERR = 20030;
14     public static final Integer GET_ERR = 20040;
15 }
```

### 三、重构 BookController

```
1  package com.zsh.controller;
2
3  import com.zsh.domain.Book;
4  import com.zsh.service.BookService;
5  import org.springframework.beans.factory.annotation.Autowired;
6  import org.springframework.web.bind.annotation.*;
7
8  import java.util.List;
9
10 @RestController
11 @RequestMapping("/books")
12 public class BookController {
13
14     @Autowired
15     private BookService bookService;
16
17     @PostMapping
18     public Result save(@RequestBody Book book) {
19         boolean flag = bookService.save(book);
20         return new Result(flag ? Code.SAVE_OK : Code.SAVE_ERR, flag);
21     }
22
23     @DeleteMapping("/{id}")
24     public Result delete(@PathVariable Integer id) {
25         boolean flag = bookService.delete(id);
26         return new Result(flag ? Code.DELETE_OK : Code.DELETE_ERR,
27 flag);
28     }
29
30     @PutMapping
31     public Result update(@RequestBody Book book) {
32         boolean flag = bookService.update(book);
```



```
32         return new Result(flag ? Code.UPDATE_OK : Code.UPDATE_ERR,
33         flag);
34     }
35     @GetMapping("/{id}")
36     public Result getById(@PathVariable Integer id) {
37         Book book = bookService.getById(id);
38         Integer code = book != null ? Code.GET_OK : Code.GET_ERR;
39         String msg = book != null ? "数据查询成功" : "数据查询失败，请重
40         试";
41         return new Result(code, book, msg);
42     }
43     @GetMapping
44     public Result getAll() {
45         List<Book> books = bookService.getAll();
46         Integer code = books != null ? Code.GET_OK : Code.GET_ERR;
47         String msg = books != null ? "数据查询成功" : "数据查询失败，请重
48         试";
49         return new Result(code, books, msg);
50     }
51 }
```

## PartⅢ 项目异常处理

### 一、异常处理器

#### 1.异常的常见位置和诱因

| 常见位置   | 诱因               |
|--------|------------------|
| 框架内部异常 | 使用不合规            |
| 数据层异常  | 外部服务器故障，如服务器访问超时 |

| 常见位置  | 诱因                           |
|-------|------------------------------|
| 业务层异常 | 业务逻辑书写错误，如遍历业务书写操作导致索引越界异常   |
| 表现层异常 | 数据收集、校验等规则错误，如不匹配的数据类型间导致异常  |
| 工具类异常 | 工具类书写不严谨不够健壮，如长期未释放某个必须释放的连接 |

## 2.异常处理思路

各个层级均有可能出现异常，我们将所有异常都抛出到表现层进行统一处理。

表现层处理异常，若每个方法中单独书写，代码臃肿且意义不大，如何解决？——**AOP**思想。

## 3.异常处理器

```
1 package com.zsh.controller;
2
3 import org.springframework.web.bind.annotation.ExceptionHandler;
4 import org.springframework.web.bind.annotation.RestControllerAdvice;
5
6 /**
7  * 项目异常统一处理
8  */
9 @RestControllerAdvice// REST风格
10 public class ProjectExceptionHandler {
11
12     @ExceptionHandler(Exception.class)// 拦截所有异常并处理
13     public Result handleException(Exception e) {
14         System.out.println("catch an exception...");
15         return new Result(666, null, "catch an exception");
16     }
17 }
```

## 二、项目异常处理方案

### 1.项目异常分类

- **业务异常(BusinessException):**
  - 规范的用户行为操作产生的异常，如用户在年龄输入框输入了“十八”而非“18”。。
  - 不规范的用户行为操作产生的异常，如用户在地址栏输入了 `http://localhost/users/heihei` 而非 `http://localhost/books/1`。
- **系统异常(SystemException):** 项目运行过程中可预计且无法避免的异常，如服务器宕机。
- **其他异常(Exception):** 编程人员未预料到的异常，如找不到指定路径的文件。

### 2.处理方案

- **业务异常:** 发送**对应**消息给用户，提醒用户进行规范操作。
- **系统异常:**
  - 发送**固定**消息给用户，安抚用户。
  - 发送**特定**消息给运维人员，提醒他们维护。
  - 记录日志。
- **其他异常:**
  - 发送**固定**消息给用户，安抚用户。
  - 发送**特定**消息给开发人员，提醒他们将此异常纳入预期范围内。
  - 记录日志。

### 3.代码实现

#### 3.1 定义 `BusinessException` 和 `SystemException`

- `BusinessException`:

```
1 package com.zsh.exception;
```

```

2
3 public class BuisnessException extends RuntimeException {
4     private Integer code;
5
6     public BuisnessException(Integer code, String message) {
7         super(message);
8         this.code = code;
9     }
10
11     public BuisnessException(Integer code, String message,
12 Throwable cause) {
13         super(message, cause);
14         this.code = code;
15     }
16
17     public Integer getCode() {
18         return code;
19     }
20
21     public void setCode(Integer code) {
22         this.code = code;
23     }
24 }

```

- **SystemException:**

```

1 package com.zsh.exception;
2
3 public class SystemException extends RuntimeException {
4     private Integer code;
5
6     public SystemException(Integer code, String message) {
7         super(message);
8         this.code = code;
9     }
10
11     public SystemException(Integer code, String message, Throwable
12 cause) {
13         super(message, cause);
14         this.code = code;
15     }
16
17     public Integer getCode() {
18         return code;
19     }
20 }

```

```

18     }
19
20     public void setCode(Integer code) {
21         this.code = code;
22     }
23 }

```

## 3.2 更新 Code 类

```

1 package com.zsh.controller;
2
3 public class Code {
4     // 结尾1代表成功，0代表失败
5
6     public static final Integer SAVE_OK = 20011;
7     public static final Integer DELETE_OK = 20021;
8     public static final Integer UPDATE_OK = 20031;
9     public static final Integer GET_OK = 20041;
10
11     public static final Integer SAVE_ERR = 20010;
12     public static final Integer DELETE_ERR = 20020;
13     public static final Integer UPDATE_ERR = 20030;
14     public static final Integer GET_ERR = 20040;
15
16     public static final Integer SYSTEM_ERR = 50001;
17     public static final Integer SYSTEM_TIMEOUT_ERR = 50002;
18     public static final Integer BUSINESS_ERR = 60001;
19     public static final Integer UNKNOWN_ERR = 99999;
20 }

```

## 3.3 在 BookServiceImpl 中触发自定义异常

```

1 package com.zsh.service.impl;
2
3 import com.zsh.controller.Code;
4 import com.zsh.dao.BookDao;
5 import com.zsh.domain.Book;

```

```
6 import com.zsh.exception.BuisnessException;
7 import com.zsh.exception.SystemException;
8 import com.zsh.service.BookService;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.stereotype.Service;
11
12 import java.util.List;
13
14 @Service
15 public class BookServiceImpl implements BookService {
16
17     @Autowired
18     private BookDao bookDao;
19
20     @Override
21     public boolean save(Book book) {
22         bookDao.save(book);
23         return true;
24     }
25
26     @Override
27     public boolean delete(Integer id) {
28         bookDao.delete(id);
29         return true;
30     }
31
32     @Override
33     public boolean update(Book book) {
34         bookDao.update(book);
35         return true;
36     }
37
38     @Override
39     public Book getById(Integer id) {
40         if (id < 1) {
41             throw new BuisnessException(Code.BUSINESS_ERR, "您输入的id不
合法，请重新输入!");
42         }
43
44         try {
45             int i = 1 / 0; // 模拟异常
46         } catch (Exception e) {
47             throw new SystemException(Code.SYSTEM_TIMEOUT_ERR, "服务器访
问超时，请重试!", e);
48         }
49     }
50 }
```

```

49         return bookDao.getById(id);
50     }
51
52     @Override
53     public List<Book> getAll() {
54         return bookDao.getAll();
55     }
56 }

```

### 3.4 重构 ProjectExceptionHandler

```

1  package com.zsh.controller;
2
3  import com.zsh.exception.BuisnessException;
4  import com.zsh.exception.SystemException;
5  import org.springframework.web.bind.annotation.ExceptionHandler;
6  import org.springframework.web.bind.annotation.RestControllerAdvice;
7
8  /**
9   * 项目异常统一处理
10  */
11  @RestControllerAdvice// REST风格
12  public class ProjectExceptionHandler {
13
14      @ExceptionHandler(SystemException.class)// 拦截系统异常并处理
15      public Result handleSystemException(SystemException e) {
16          // 1.记录日志
17          // 2.发送消息给运维人员
18          // 3.发送邮件给开发人员
19          return new Result(e.getCode(), null, e.getMessage());
20      }
21
22      @ExceptionHandler(BuisnessException.class)// 拦截业务异常并处理
23      public Result handleBusinessException(BuisnessException e) {
24          return new Result(e.getCode(), null, e.getMessage());
25      }
26
27      @ExceptionHandler(Exception.class)// 拦截其他异常并处理
28      public Result handleException(Exception e) {
29          // 1.记录日志
30          // 2.发送消息给运维人员

```

```
31         // 3.发送邮件给开发人员
32         return new Result(Code.UNKNOWN_ERR, null, "系统繁忙，请稍后再
    试!");
33     }
34 }
```

---

## PartIV 前后端协议联调